

Data Analysis with Python 2024–2025

Parte 2: Python Conteúdo Teórico

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**.

O que você vai aprender nesta página

- Compreender o funcionamento das expressões regulares em Python.
- Aprender a processar arquivos em Python.
- Entender os conceitos de objetos e classes.
- Saber tratar exceções em um programa Python.

Conteúdo

1 Expressões regulares

Já vimos que é possível perguntar a uma string `str` se ela começa com um determinado prefixo usando `str.startswith('Apple')`. Se quiséssemos verificar se ela começa com `"Apple"` ou `"apple"`, teríamos que chamar o método duas vezes. As expressões regulares oferecem uma forma mais simples de resolver isso: `re.match(r"[Aa]pple", str)`. A notação entre colchetes é um exemplo da sintaxe especial das expressões regulares: ela indica que qualquer um dos caracteres dentro dos colchetes é aceito – neste caso, `"A"` ou `"a"`. O restante dos caracteres em `"pple"` funciona normalmente. A string `r"[Aa]pple"` é chamada de *padrão* (pattern).

Um exemplo um pouco mais elaborado verifica se a string começa com `apple` ou `banana` (independentemente de maiúsculas ou minúsculas na primeira letra): `re.match(r"[Aa]pple|[Bb]anana", str)`. Aqui aparece um novo caractere especial, a barra vertical `|`, que representa uma alternativa. De cada lado da barra temos um subpadrão.

Um nome de variável válido em Python começa com uma letra ou um sublinhado, e os caracteres seguintes também podem ser dígitos. Assim, `_oculto`, `L_valor` e `A123_` são nomes válidos, mas `2abc` não é. O padrão de expressão regular para reconhecer nomes de variáveis válidos seria: `r"[A-Za-z_][A-Za-z_0-9]*\Z"`. Aqui usamos uma abreviação para intervalos de caracteres: `A-Z` significa todos os caracteres de `A` até `Z`.

O primeiro caractere do nome é definido no primeiro par de colchetes; os caracteres seguintes, no segundo par. O caractere especial `*` significa “qualquer quantidade (0, 1, 2, ...) do subpadrão anterior”. Por exemplo, o padrão `r"ba*"` aceita as strings `"b"`, `"ba"`, `"baa"`, `"baaa"` e assim por diante. A notação especial `\Z` indica o fim da string. Sem ela, uma string como `abc-` também seria aceita, pois a função `match` normalmente verifica apenas se a string *começa* com o padrão.

As notações especiais, como `\Z`, podem gerar confusão, já que os literais de string em Python também usam notações parecidas: `\n` representa uma quebra de linha, `\t` representa uma tabulação, e assim por diante. Isso é resolvido usando as chamadas *raw strings* (strings “cruas”), indicadas por um `r` antes das aspas, por exemplo `r"ab*\Z"`. Em uma raw string, notações como `\n` e `\t` não são interpretadas. **É recomendável sempre usar raw strings ao definir padrões de expressões regulares!**

1.1 Padrões

Um padrão representa um conjunto de strings – potencialmente infinito – que compartilham alguma característica em comum. Expressões regulares (RE) são um tópico clássico da ciência da computação e são muito comuns em tarefas de programação. Linguagens de script, como Python, lidam muito bem com expressões regulares, permitindo processar textos de forma bem sofisticada.

Em um padrão, caracteres normais (letras, números) representam a si mesmos, a menos que sejam precedidos por uma barra invertida, o que pode ativar um significado especial. Sinais de pontuação têm significado especial, a não ser que sejam precedidos por `\`, o que remove esse significado especial. Use `\\` para representar uma barra invertida literal.

Símbolo	Significado
.	Casa com qualquer caractere
[...]	Casa com qualquer caractere presente dentro dos colchetes
[^...]	Casa com qualquer caractere que <i>não</i> apareça após o acento circunflexo
^	Casa com o início da string
\$	Casa com o final da string
*	Casa com zero ou mais ocorrências da RE anterior
+	Casa com uma ou mais ocorrências da RE anterior
{m,n}	Casa de m a n ocorrências da RE anterior
?	Casa com zero ou uma ocorrência da RE anterior

Já vimos que o caractere `|` denota alternativas. Por exemplo, o padrão `r"Get (on|off|ready)"` casa com as strings `"Get on"`, `"Get off"` e `"Get ready"`. Parênteses podem ser usados para criar agrupamentos dentro de um padrão: `r"(ab)+"` casa com `"ab"`, `"abab"`, `"ababab"`, etc. Esses grupos recebem também um número de referência, a partir de 1, e podem ser referenciados dentro do próprio padrão por meio de *retro-referências* (`\número`). Por exemplo, podemos localizar padrões repetidos separados por espaço com `r"([a-z]{3,}) \1 \1"`, o que reconhece strings como `"aca aca aca"` ou `"turn turn turn"`, mas não `"aca aba aca"` nem `"ac ac ac"`.

Um acento circunflexo como primeiro caractere dentro de colchetes cria o *conjunto complementar* de caracteres:

Símbolo	Significado
<code>\d</code>	equivalente a <code>[0-9]</code> , casa com um dígito
<code>\D</code>	equivalente a <code>[^0-9]</code> , casa com qualquer coisa exceto um dígito
<code>\s</code>	casa com um caractere de espaço em branco (espaço, quebra de linha, tabulação, ...)
<code>\S</code>	casa com um caractere que não é espaço em branco
<code>\w</code>	equivalente a <code>[a-zA-Z0-9_]</code> , casa com um caractere alfanumérico
<code>\W</code>	casa com um caractere não alfanumérico

Com essa notação, o exemplo de nome de variável pode ser reescrito de forma mais curta como `r'[a-zA-Z_]\w*\Z'`.

Os padrões `\A`, `\b`, `\B` e `\Z` casam todos com uma string vazia, mas em posições específicas. `\A` e `\Z` reconhecem, respectivamente, o início e o fim da string inteira – diferentemente de `^` e `$`, que em certos modos também podem casar após ou antes de uma quebra de linha. O padrão `\b` casa com o início ou o fim de uma palavra; `\B` faz o oposto.

1.2 Funções de busca

Até agora usamos apenas a função `re.match`, que tenta encontrar uma correspondência no *início* da string. A função `re.search` permite localizar a correspondência em qualquer parte da string. Por exemplo, `re.search(r'\bback\b', s)` casa com `"back"`, `"a back, is a body part"` e `"get back"`, mas não com `"backspace"` ou `"comeback"`.

A função `re.search` encontra apenas a primeira ocorrência. Podemos usar `re.findall` para encontrar todas as ocorrências. Suponha que queremos encontrar todos os gerúndios (palavras terminadas em `'ing'`) em uma string `s`:

```
import re
s = "Doing things, going home, staying awake, sleeping later"
re.findall(r'\w+ing\b', s)
# ['Doing', 'going', 'staying', 'sleeping']
```

Para capturar todos os números inteiros de uma string, podemos usar `re.findall(r'[+-]?\d+', s)`:

```
re.findall(r'[+-]?\d+', "23 + -24 = -1")
# ['23', '-24', '-1']
```

1.2.1 Casamento guloso e não guloso

Suponha que temos frases do tipo “se ..., então ...” e queremos extrair apenas a condição. Uma primeira tentativa, `re.findall(r'[Ii]f (.*) , then', s)`, tende a capturar texto demais, pois o padrão `.*` tenta casar o máximo possível de caracteres – esse comportamento é chamado de *casamento guloso* (*greedy matching*).

Uma forma de resolver isso é impedir que o ponto final seja incluído no trecho capturado, usando uma classe de caracteres complementar: `[^.]`, resultando em `re.findall(r'[Ii]f ([^.]*), then', s)`.

Outra solução é usar o *casamento não guloso*. Os quantificadores `+`, `*`, `?` e `{m,n}` possuem versões não gulosas: `+`, `*`, `?` e `{m,n}?`. Essas versões tentam usar o *mínimo* de caracteres possível para que todo o padrão ainda case com algum trecho da string:

```
re.findall(r'[Ii]f (.*)', then', s)
```

1.3 Funções do módulo re

As funções mais comuns do módulo `re` são:

- `re.match(padrao, str)`
- `re.search(padrao, str)`
- `re.findall(padrao, str)`
- `re.finditer(padrao, str)`
- `re.sub(padrao, substituicao, str, count=0)`

As funções `match` e `search` retornam um *objeto de correspondência* (*match object*), que descreve a ocorrência encontrada. A função `findall` retorna uma lista com todas as ocorrências (como strings). A função `finditer` funciona como `findall`, mas retorna um iterador cujos itens são objetos de correspondência. A função `sub` substitui todas as ocorrências do padrão em `str` pela string de substituição e retorna a nova string.

```
import re
frase = "She goes where she wants to, she's a sheriff."
nova_frase = re.sub(r'\b[Sh]he\b', 'he', frase)
print(nova_frase)
# he goes where he wants to, he's a sheriff.
```

A função `sub` também aceita retro-referências para se referir ao texto casado. As retro-referências `\1`, `\2` etc. referem-se, em ordem, aos grupos do padrão:

```
import re
texto = """He is a timelord.
He has a Tardis."""
novo_texto = re.sub(r'(\b[Hh]e\b)', r'\1 (The Doctor)', texto, 1)
print(novo_texto)
# He (The Doctor) is a timelord.
# He has a Tardis.
```

1.4 O objeto de correspondência (*match object*)

As funções `match`, `search` e `finditer` usam objetos de correspondência para descrever a ocorrência encontrada. O método `groups()` de um objeto de correspondência retorna uma tupla com todas as substrings casadas pelos grupos do padrão. Cada par de parênteses no padrão cria um novo grupo, referenciado pelos índices 1, 2, ... O grupo 0 é especial: representa o casamento produzido pelo padrão inteiro.

```
mo = re.search(r'\d+ (\d+) \d+ (\d+)', 'first 123 45 67 890 last')
mo.groups()      # ('45', '890')
mo.group(1)      # '45'
mo.group(0)      # '123 45 67 890'
```

Além de acessar as strings casadas pelo padrão e seus grupos, também é possível obter os índices correspondentes na string original:

- `start(gid=0)` e `end(gid=0)` retornam, respectivamente, os índices de início e fim do grupo `gid`;
- `span(gid)` retorna o par (início, fim).

O objeto de correspondência também pode ser usado como um valor booleano:

```
mo = re.search(...)
if mo:
    # faça algo
```

Ou, de forma explícita, `encontrado = bool(mo)`.

1.5 Outras informações úteis

Quando o mesmo padrão é usado em várias chamadas, pode ser interessante *pré-compilá-lo* por questões de eficiência, usando a função `compile(padrao, flags=0)` do módulo `re`. Essa função retorna um chamado *objeto RE*, que possui versões dos métodos do módulo `re`, com a diferença de que o padrão já está armazenado no objeto (não é mais o primeiro parâmetro).

Detalhes do casamento podem ser ajustados por *flags* opcionais, fornecidas dentro do próprio padrão ou como segundo parâmetro da função `compile`:

Notação no padrão	Flag
(?i)	<code>re.IGNORECASE</code>
(?m)	<code>re.MULTILINE</code>
(?s)	<code>re.DOTALL</code>

A flag `IGNORECASE` faz com que letras maiúsculas e minúsculas sejam tratadas como equivalentes. A flag `MULTILINE` faz com que `^` e `$` casem com o início e o fim de *cada linha*, além do início e fim da string inteira – isso é o que diferencia `\A` de `^`, e `\Z` de `$`. A flag `DOTALL` faz com que o ponto (`.`) também aceite o caractere de quebra de linha. Múltiplas flags podem ser combinadas com `|`: `re.compile(padrao, re.MULTILINE | re.DOTALL)`.

2 Processamento básico de arquivos

Um arquivo pode ser aberto com a função `open`. A chamada `open(nome_do_arquivo, mode="r")` retorna um objeto de arquivo, usado para referenciar um arquivo em disco. Para ler ou escrever, usamos os métodos `read` e `write` desse objeto. Ao terminar de usá-lo, deve-se chamar o método `close`.

O parâmetro `mode` controla que tipo de operação pode ser feita no arquivo:

Modo	Descrição
<code>r</code>	somente leitura; o arquivo precisa existir
<code>w</code>	somente escrita; cria o arquivo, ou sobrescreve se já existir
<code>a</code>	somente escrita; a escrita sempre ocorre no final do arquivo (append)
<code>r+</code>	leitura e escrita; o arquivo precisa existir
<code>w+</code>	leitura e escrita; cria, ou sobrescreve se já existir
<code>a+</code>	leitura e escrita; a escrita ocorre no final do arquivo

Ao final da string de modo, pode-se acrescentar a letra `t` (modo texto) ou `b` (modo binário); se omitida, o padrão é o modo texto.

No modo binário, o conteúdo do arquivo não é interpretado de forma alguma: os métodos `read` e `write` lidam com *bytes* (um byte tem 8 bits e pode representar um número entre 0 e 255).

No modo texto, duas conversões ocorrem automaticamente:

- No Windows, o fim de linha é codificado por dois caracteres; ao ler o arquivo, esses dois caracteres são convertidos para `'\n'`, e o processo inverso ocorre ao escrever.
- Cada caractere é codificado no arquivo como um ou mais bytes. Uma codificação muito usada é a UTF-8. Nessa codificação, por exemplo, o caractere `'ã'` é representado por dois bytes.

Neste curso trabalharemos apenas com arquivos de texto, então o modo texto padrão é suficiente. Ainda assim, às vezes é necessário especificar explicitamente a codificação de um arquivo, caso não seja UTF-8.

2.1 Métodos comuns do objeto de arquivo

- `read(tamanho)`: lê `tamanho` caracteres/bytes como string.
- `write(string)`: escreve uma string/bytes no arquivo.
- `readline()`: lê uma linha, incluindo o caractere de quebra de linha final.
- `readlines()`: retorna uma lista com todas as linhas do arquivo.
- `writelines()`: escreve uma lista de linhas no arquivo.
- `flush()`: tenta garantir que as alterações sejam gravadas em disco imediatamente.

```
f = open("caderno.ipynb", "r") # abre um arquivo de texto
for i in range(5):             # le as cinco primeiras linhas
    linha = f.readline()
    print(f"Linha {i}: {linha}", end="")
f.close()
```

É fácil esquecer de fechar um arquivo. Uma solução é usar um *gerenciador de contexto*, criado com a instrução `with`: ao sair do bloco indentado, o arquivo é fechado automaticamente.

```
with open("caderno.ipynb", "r") as f: # o arquivo sera fechado
    for i in range(5):                 # automaticamente ao sair do
        bloco                          bloco
        linha = f.readline()
        print(f"Linha {i}: {linha}", end="")
```

O objeto de arquivo é iterável, ou seja, podemos percorrer as linhas do arquivo usando um laço `for`:

```
maior_tamanho = 0
with open("caderno.ipynb", "r") as f:
    for linha in f:                    # percorre todas as linhas do
        arquivo                       arquivo
        if len(linha) > maior_tamanho:
            maior_tamanho = len(linha)
print(f"A maior linha deste arquivo tem tamanho {maior_tamanho}")
```

2.2 Objetos padrão de entrada e saída

O Python mantém três objetos de arquivo automaticamente abertos:

- `sys.stdin` – entrada padrão;
- `sys.stdout` – saída padrão;
- `sys.stderr` – saída padrão de erro.

Para ler uma linha digitada pelo usuário, usa-se `sys.stdin.readline()`. Para escrever uma linha na tela, usa-se `sys.stdout.write(linha)`. A saída de erro deve ser usada apenas para mensagens de erro, ainda que, muitas vezes, seu destino final seja o mesmo da saída padrão.

A função `print` usa o arquivo `sys.stdout`, e a função `input` usa o arquivo `sys.stdin`. Esses objetos podem ter seu destino redirecionado – é comum, por exemplo, redirecionar `stderr` para um arquivo de log.

2.3 O módulo `sys`

Além dos três objetos de arquivo padrão, o módulo `sys` tem outros atributos úteis. `sys.path` é a lista de pastas em que o Python procura por módulos a serem importados. `sys.argv` contém os chamados *parâmetros de linha de comando*: ao executar `python3 programa.py param1 param2 ...` no terminal, o nome do programa fica em `sys.argv[0]`, e os demais parâmetros ficam nas posições seguintes da lista.

A função `sys.exit` pode ser usada para encerrar imediatamente o programa. O parâmetro inteiro fornecido é o valor de retorno do programa – por convenção, 0 indica sucesso, e qualquer valor diferente de zero indica que ocorreu um erro.

3 Objetos e classes

Python é uma linguagem orientada a objetos, como Java e C++, mas, diferentemente de Java, não obriga o uso de classes, herança e métodos: também é possível programar de forma estrutural, com funções e módulos.

Todo valor em Python é um objeto. Objetos combinam dados e as funções que manipulam esses dados – essa combinação é chamada de *encapsulamento*. Os itens de dados e as funções de um objeto são chamados de *atributos*; em particular, os atributos que são funções são chamados de *métodos*. Por exemplo, o operador `+` em números inteiros invoca um método dos inteiros, e o operador `+` em strings invoca um método das strings.

Funções, módulos, métodos, classes etc. são todos *objetos de primeira classe* em Python, ou seja, podem ser:

- armazenados em um contêiner;
- passados como parâmetro para uma função;
- retornados por uma função;
- associados a uma variável.

Acessamos um atributo de um objeto usando o *operador ponto*: `objeto.atributo`. Por exemplo, se `L` é uma lista, referenciamos o método `append` com `L.append`, e podemos chamá-lo assim: `L.append(4)`. Como módulos também são objetos em Python, a expressão `math.pi` nada mais é do que o acesso ao atributo de dado `pi` do objeto módulo `math`.

Números como 2 e 100 são instâncias do tipo `int`. Da mesma forma, "ola" é uma instância do tipo `str`. Quando escrevemos `s = set()`, estamos, na verdade, criando uma nova instância do tipo `set` e associando esse objeto à variável `s`.

O usuário pode definir seus próprios tipos de dado – as chamadas *classes*. Ao chamar uma classe como se fosse uma função, obtemos um novo objeto *instância* desse tipo. Classes podem ser vistas como “receitas” para criar objetos.

```
class MinhaClasse(object):
    """String de documentacao da classe"""

    def __init__(self, param1, param2):
        """Inicializa uma instancia de MinhaClasse"""
        self.b = param1      # cria um atributo de instancia
        c = param2           # cria uma variavel local da funcao
        # demais instrucoes ...

    def f(self, param1):
        """Este e um metodo da classe"""
        # algumas instrucoes

a = 1    # isto cria um atributo de classe
```

A definição de classe começa com a palavra-chave `class`. Nela damos um nome ao novo tipo e, entre parênteses, listamos as classes-base. O bloco indentado seguinte é o *corpo da classe*. Depois de todo o corpo ser lido, um novo tipo é criado – note que nenhuma instância é criada ainda. Todos os atributos e métodos da classe são definidos no corpo da classe.

A classe do exemplo tem dois métodos: `__init__` e `f`. O primeiro parâmetro deles é especial: `self` refere-se à instância da classe sobre a qual o método está sendo chamado. Por exemplo, se criarmos `a = MinhaClasse(...)`, então, na chamada `a.f(param1)`, `self` representa `a`. O conceito de `self` é equivalente ao de `this` em C++ ou Java.

O método `__init__` realiza a inicialização quando uma instância é criada. Ao instanciar com `i = MinhaClasse(2,3)`, os parâmetros `param1` e `param2` recebem os valores 2 e 3, respectivamente. Com a instância `i` criada, podemos chamar seu método `f` com o operador ponto: `i.f(1)`.

Há diferenças em como uma atribuição dentro do corpo da classe cria variáveis. O atributo `a` é definido no nível da classe e é comum a todas as instâncias de `MinhaClasse`. A variável `c` é local à função `__init__` e não pode ser usada fora dela. Já o atributo `b` é específico de cada instância. Assim, para objetos `x = MinhaClasse(1,0)` e `y = MinhaClasse(2,0)`, temos `x.b != y.b`, mas `x.a == y.a`.

Todo método de uma classe recebe obrigatoriamente um primeiro parâmetro que se refere à instância na qual o método foi chamado, chamado convencionalmente de `self`. Para acessar um atributo de classe a partir de um método, use a forma completa `MinhaClasse.a`. Métodos cujo nome começa e termina com dois sublinhados são chamados de *métodos especiais*; `__init__` é um deles.

3.1 Instâncias

Criamos instâncias chamando uma classe como se fosse uma função: `i = NomeDaClasse(...)`. Os parâmetros fornecidos na chamada são passados para a função `__init__`, onde os atributos específicos da instância podem ser criados. Se `__init__` estiver ausente, é possível criar uma instância sem fornecer nenhum parâmetro – nesse caso, ela não terá atributos. Mais tarde, é possível (re)associar atributos com a atribuição `instancia.atributo = novo_valor`.

Se o atributo não existia antes, ele é adicionado à instância com o valor atribuído. Em Python é realmente possível adicionar ou remover atributos de uma instância já existente, porque

os nomes dos atributos e seus respectivos valores são armazenados em um dicionário, que é, ele mesmo, um atributo da instância chamado `__dict__`. Outro atributo padrão, além de `__dict__`, é `__class__`, que armazena a classe da instância – ou seja, o tipo do objeto.

3.2 Busca de atributos

Suponha que `x` é uma instância da classe `X`, e queremos ler um atributo `x.a`. A busca ocorre em três etapas:

1. verifica-se primeiro se `a` é um atributo da instância `x`;
2. caso não seja, verifica-se se `a` é um atributo de classe de `X`;
3. caso ainda não seja, as classes-base de `X` são verificadas.

Já para *associar* um atributo, as regras são mais simples: `x.a = valor` define o atributo de instância, e `X.a = valor` define o atributo de classe. Se uma classe-base, a própria classe `X` e a instância `x` tiverem, cada uma, um atributo chamado `a`, então `x.a` esconde `X.a`, que por sua vez esconde o atributo da classe-base.

3.3 Herança

A herança nos permite reaproveitar o código de uma classe já existente `B` na criação de uma nova classe `C`. Relembrando: ao procurar um atributo, a busca começa no dicionário da instância e continua pelos atributos de classe; se ambos falharem, o atributo é buscado nas classes-base e, recursivamente, nas classes-base delas.

Assim, pode parecer que estamos acessando um atributo da classe `C`, quando na verdade estamos acessando um atributo herdado de sua classe-base `B`. Nesse caso, dizemos que `C` *herda* o atributo de sua classe-base `B`. Se houver atributos com o mesmo nome tanto na classe quanto na sua classe-base, o atributo da classe-base fica oculto – dizemos que a classe `C` *sobrescreve* (*override*) o atributo da classe-base `B`. Terminologia: `B` é a classe-base (superclasse) e `C` é a classe derivada (subclasse).

```
class B(object):
    def f(self):
        print("Executando B.f")
    def g(self):
        print("Executando B.g")

class C(B):
    def g(self):
        print("Executando C.g")

x = C()
x.f()    # herdado de B
x.g()    # sobrescrito por C
# Saída:
# Executando B.f
# Executando C.g
```

A relação de herança entre duas classes `B` e `C` pode ser testada com a função `issubclass`: `issubclass(C, B) == True`, mas `issubclass(B, C) == False`. A função `isinstance(obj, cls)` permite verificar se uma instância tem tipo `cls` ou algum ancestral do tipo `cls`. Criando `x = C()` e `y = B()`, temos `isinstance(x, B) == isinstance(x, C) == isinstance(y, B) == True`, mas `isinstance(y, C) == False`.

A classe `object` deve ser uma classe-base ou ancestral de toda classe em Python: para toda instância `x`, `isinstance(x, object) == True`.

Ao derivar de uma classe existente, podemos modificar e/ou estender seu comportamento sem alterar a classe original. Por exemplo, se quisermos adicionar um método à classe `list`, podemos usar herança e codificar apenas a parte que muda. Outro uso da herança é criar hierarquias conceituais – como veremos na hierarquia de exceções do Python. Um terceiro uso é criar interfaces: várias classes podem compartilhar a mesma interface (isto é, oferecer os mesmos atributos), mas ter comportamentos ou implementações muito diferentes, o que permite trocar parte do programa com mínimas alterações no restante do código.

Se, na definição de um método `C.f`, precisarmos chamar o método correspondente da classe-base `B`, podemos usar a chamada completa `B.f(...)`. Isso é chamado de *delegação*, útil, por exemplo, para chamar o `__init__` da classe-base a partir do `__init__` da classe derivada, inicializando os atributos da classe-base.

3.4 Métodos especiais

Já conhecemos um método especial, o `__init__`, que define os valores iniciais dos atributos de instância. A seguir, apresentamos os principais métodos especiais de propósito geral, executados quando certas operações são realizadas sobre os objetos.

Seja `C` uma classe e `x, y` suas instâncias: `__hash__` retorna um valor inteiro, com o requisito de que `x == y` implica `x.__hash__() == y.__hash__()`. O valor é usado ao armazenar objetos em dicionários e conjuntos; as instâncias devem ser imutáveis. Uma classe com o método `__call__` torna suas instâncias “chamáveis”: a chamada `x(a, b, ...)` invocará esse método especial com os parâmetros fornecidos. O método `__del__` é chamado quando a instância correspondente é destruída. O método `__new__` controla a criação de novas instâncias, podendo ser usado, por exemplo, para criar classes que possuem apenas uma instância.

O método `__str__` é chamado quando `print` precisa exibir o valor de uma instância – deve retornar uma string. O método `__repr__` é chamado quando o interpretador interativo exibe o valor de uma expressão avaliada; retorna uma representação “canônica”, que (ao menos em teoria) poderia ser usada para recriar o objeto original. Os métodos especiais `__eq__`, `__ge__`, `__gt__`, `__le__`, `__lt__` e `__ne__` são chamados quando os operadores correspondentes `==`, `>=`, `>`, `<=`, `<` e `!=` são usados.

Para que instâncias de uma classe suportem operações numéricas (+, -, *, / etc.), é preciso definir os métodos especiais correspondentes. Por exemplo, a expressão `x + y` resulta na chamada `x.__add__(y)`, que deve retornar o resultado da operação.

Método	Operação
<code>__add__</code>	adição (+)
<code>__sub__</code>	subtração (-)
<code>__mul__</code>	multiplicação (*)
<code>__truediv__</code>	divisão (/)
<code>__floordiv__</code>	divisão inteira (//)

As atribuições compostas correspondentes (`+=`, `-=`, `*=`, `/=`) usam os métodos especiais `__iadd__`, `__isub__`, `__imul__` e `__itruediv__`. As funções de conversão `complex()`, `float()` e `int()` chamam, respectivamente, `__complex__`, `__float__` e `__int__`.

Além dos métodos habituais dos contêineres, como `append` de uma lista, várias operações são resolvidas por chamadas a métodos especiais da classe do contêiner. O teste de pertencimento `x in c` corresponde à chamada `c.__contains__(x)`. A exclusão de um elemento com `del c[chave]` corresponde a `c.__delitem__(chave)`. A leitura de um item com `c[chave]` corresponde a `c.__getitem__(chave)`, e a atribuição `c[chave] = valor` corresponde

a `c.__setitem__(chave, valor)`. O número de elementos de um contêiner `c` é obtido com `len(c)`, que na verdade chama `c.__len__()`. A chamada `iter(c)` invoca o método especial `__iter__`.

4 Exceções

Quando ocorre um erro, o que podemos fazer?

- Exibir uma mensagem de erro;
- Interromper a execução do programa;
- Sinalizar o erro retornando um valor especial, como `-1` ou `None`;
- Ignorar o erro;
- ...

Essas soluções tendem a misturar a *indicação* do problema com a *reação* a esse problema. O comportamento do programa diante de uma situação de erro fica fixo na implementação da função, e a instância que chamou a função nem sempre sabe o que fazer diante da situação.

A maioria das linguagens modernas possui um sistema chamado *tratamento de exceções*, que separa o reconhecimento de erros do seu tratamento propriamente dito. Sinalizamos um erro ou situação anômala *lançando* (*raising*) uma exceção, o que em Python é feito com a instrução `raise`:

- `raise instancia`
- `raise classe_de_excecao[, expressao]`

Na segunda forma, se a expressão existir, ela é uma tupla de parâmetros passados para a classe de exceção.

As funções da biblioteca padrão do Python lançam exceções em situações de erro. Às vezes, uma exceção nem representa exatamente um erro – por exemplo, quando um iterador se esgota, ele sinaliza isso lançando a exceção `StopIteration`. Outro exemplo de exceção pouco “grave” é `Warning`.

A forma geral da instrução de captura de exceções é a seguinte:

```
try:
    # instrucoes que podem causar excecoes
except (NomeExcecao1, NomeExcecao2, ...):
    # tratamento das excecoes
else:
    # executado se o bloco try nao causou nenhuma excecao
finally:
    # sempre executado -- codigo de limpeza
```

Em geral, apenas as partes `try` e `except` são necessárias.

```
L = [1, 2, 3]
try:
    print(L[3])
except IndexError:
    print("Este indice nao existe")
# Saida: Este indice nao existe
```

```
def calcula_media(L):
    n = len(L)
    s = sum(L)
    return float(s) / n    # o erro e detectado aqui!

minha_lista = []
while True:
    try:
        x = float(input("Digite um numero (nao numerico encerra): "))
        minha_lista.append(x)
    except ValueError:
        break

try:
    media = calcula_media(minha_lista)
    print("A media e", media)
except ZeroDivisionError:
    # e o erro e tratado aqui
    if len(minha_lista) == 0:
        print("Tentou calcular a media de uma lista vazia de numeros")
    else:
        print("Algo estranho aconteceu")
```

4.1 A hierarquia de exceções

Em Python, exceções são objetos, como qualquer outro valor, instanciados a partir de classes de exceção. As classes de exceção formam hierarquias naturais:

- novas classes de exceção podem ser criadas herdando de classes de exceção já existentes e estendendo-as;
- a raiz dessa hierarquia é a classe `Exception`;
- o Python define diversas classes-base para servir de ponto de partida, além de várias classes de exceção prontas para uso.

Algumas das classes mais comuns e seus relacionamentos:

Classe-base	Algumas classes derivadas
<code>Exception</code>	<code>SystemExit</code> , <code>StandardError</code> , <code>StopIteration</code> , <code>Warning</code>
<code>StandardError</code>	<code>NameError</code> , <code>SyntaxError</code> , <code>ArithmeticError</code> , <code>LookupError</code> , <code>EnvironmentError</code> , ...
<code>ArithmeticError</code>	<code>FloatingPointError</code> , <code>OverflowError</code> , <code>ZeroDivisionError</code>
<code>LookupError</code>	<code>IndexError</code> , <code>KeyError</code>
<code>EnvironmentError</code>	<code>IOError</code> , <code>OSError</code>

4.2 Especificações de exceção genéricas demais

A hierarquia de exceções permite capturar múltiplas exceções semelhantes capturando sua classe-base comum. Esse recurso deve ser usado com cuidado: uma especificação de exceção genérica demais, como `except Exception:`, pode esconder a verdadeira causa de um erro.

```
import sys
s = input("Digite um numero: ")
s = s[:-1]    # remove o caractere \n do final
try:
```

```
x = int(s)
sys.stdout.write(f"Voce digitou {x}\n")
except Exception:
    print("Voce nao digitou um numero")
```

Se o usuário não digitar uma string que representa um inteiro, a função `int` lança `ValueError`. Em vez de capturar `ValueError` especificamente, capturamos a raiz da hierarquia, `Exception`, o que acaba capturando *qualquer* exceção possível – inclusive um eventual erro de digitação no próprio código, que passaria despercebido. Trocar a especificação de `Exception` para `ValueError` deixaria esse tipo de problema visível.

4.3 A política de tratamento de erros do Python

O Python usa uma abordagem diferente da de muitas outras linguagens no que diz respeito à checagem de erros. Em vez de verificar previamente se todas as entradas têm o tipo correto e se o conteúdo das variáveis faz sentido para determinada operação, o Python primeiro tenta executar a operação e depois verifica se ela causou alguma exceção. Isso está relacionado ao conceito de *duck typing*: uma função funciona para um conjunto de entradas se todas as operações no corpo da função fizerem sentido para essas entradas. É por isso que os parâmetros das funções normalmente não têm um tipo especificado.

5 Sequências, iteráveis e geradores: revisão

Em termos simples, um contêiner é *iterável* se pudermos percorrer todos os seus elementos usando um laço `for`. Todas as sequências são iteráveis, mas existem outros objetos iteráveis além delas – e podemos até criar nossos próprios tipos iteráveis. Para isso, a classe precisa ter um método especial `__iter__`, que retorna um *iterador* para o contêiner. Um iterador é um objeto que possui o método `__next__`, responsável por retornar o próximo elemento do contêiner.

```
class IteradorDiaDaSemana(object):
    """Iterador sobre os dias da semana."""

    def __init__(self):
        self.i = 0      # começa na segunda-feira
        self.dias = ("Segunda", "Terca", "Quarta", "Quinta",
                     "Sexta", "Sabado", "Domingo")

    def __iter__(self):
        # se este objeto fosse um container, este metodo
        # retornaria o iterador sobre os elementos do container.
        # como este objeto ja e um iterador, retornamos self.
        return self

    def __next__(self):
        if self.i == 7:
            raise StopIteration    # sinaliza que todos os dias ja foram
                                   percorridos
        else:
            dia = self.dias[self.i]
            self.i += 1
            return dia

for dia in IteradorDiaDaSemana():
    print(dia)
```

Podemos verificar se `IteradorDiaDaSemana` é do tipo `Sequence`:

```

from collections import abc    # obtém as classes-base abstratas

containers = ["abc", [1, 2, 3], (4, 5), IteradorDiaDaSemana()]
for c in containers:
    if isinstance(c, abc.Sequence):
        print(c, "é uma sequência")
    else:
        print(c, "não é uma sequência")

```

O iterador não é uma sequência, pois, por exemplo, não é possível indexá-lo com colchetes (`[]`), mas ele é, sim, um iterável: `isinstance(IteradorDiaDaSemana(), abc.Iterable)` retorna `True`.

É possível criar iteradores manualmente, mas a sintaxe fica um tanto complicada. Uma alternativa mais simples são os *geradores*. Um gerador é uma função que contém uma instrução `yield`. Note a diferença entre geradores e expressões geradoras (vistas anteriormente no curso): ambos produzem iteráveis, mas de formas diferentes.

```

def minhas_datas(dia=1, mes=1):
    """Gera datas a partir da data informada."""
    duracoes = (31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
    primeiro_dia = dia
    for m in range(mes, 13):
        for d in range(primeiro_dia, duracoes[m - 1] + 1):
            yield (d, m)
        primeiro_dia = 1

# cria o gerador chamando a funcao:
gen = minhas_datas(26, 2)    # começa em 26 de fevereiro

for i, (dia, mes) in enumerate(gen):
    if i == 5:
        break    # imprime apenas as cinco primeiras datas do gerador
    print(f"Índice {i}, dia {dia}, mes {mes}")

```

Não seria possível escrever esse mesmo iterável usando uma expressão geradora, e seria bastante trabalhoso escrevê-lo explicitamente como um iterador, como fizemos com `IteradorDiaDaSemana`.

Referências e créditos

Este material é uma tradução e adaptação, para a língua portuguesa, do conteúdo do curso *Data Analysis with Python 2024-2025*, Capítulo 2 – “Python (Part 2)”, originalmente disponibilizado em inglês pela plataforma MOOC.fi:

<https://courses.mooc.fi/org/uh-cs/courses/data-analysis-with-python-2024-2025/chapter-2/python-continues>